



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**SMART LIBRARY MANAGEMENT, BOOK RESERVATION
AND OVERDUE NOTIFICATION SYSTEM USING JAVA**

Assignment Information

Particular	Details
Department	Artificial Intelligence and Data Science
Programme	CSE
Course Code & Name	CSA0904 – Programming in Java (SLOT A)
Academic Year / Batch	4 th year - 2027
Faculty Name	Dr. Thalapathi Rajasekaran R
Assignment Title	Smart Library Management, Book Reservation and Overdue Notification System using Java
Date of Issue	01-09-2026
Date of Submission	02-09-2026
Maximum Marks	100
Course Outcomes	CO1, CO2, CO3
Bloom's Taxonomy	L2 – Understand; L3 – Apply; L4 – Analyse
SDG Mapping	SDG 4, SDG 9, SDG 16
Industry / Societal Relevance	Digital library circulation, resource tracking, reservation management, automated notifications and reliable concurrent transaction processing

The supplied assignment identifies CO1 for object-oriented programming, CO2 for Java Collections Framework, and CO3 for multithreading and exception handling.

1. Problem Statement and Problem Formulation

1.1 Problem Statement

The objective of this project is to design, implement and evaluate a menu-driven Java application for a **Smart Library Management, Book Reservation and Overdue Notification System**.

The system manages library members, books, borrowing, returning, reservations, fines and notifications. It maintains the availability of limited book copies and ensures that concurrent requests do not create inconsistent inventory.

The application provides facilities to:

- Register members.
- Add and update books.
- Display available books and copies.
- Search books and member records.
- Issue and return books.
- Calculate overdue fines.
- Reserve unavailable books.
- Maintain reservation waitlists.
- Cancel reservations.
- Generate due-date and overdue notifications.
- Generate circulation and inventory-utilization reports.
- Handle invalid inputs and exceptional situations.
- Demonstrate multithreading, synchronization and inter-thread communication.

The supplied question specifically requires classes such as Member, Book, Reservation, Inventory and Notification, along with inheritance, polymorphism, collections, generics, iterators, exception handling and multithreading.

1.2 Problem Formulation

Inputs

- Member ID
- Member name
- Email
- Membership type

- Book ID
- Book title
- Author
- Category
- Number of copies
- Issue/return requests
- Reservation requests
- Search/update values
- Date information

Processing

1. Validate member and book details.
2. Store members and books using Java collections.
3. Check member borrowing limits.
4. Check book availability.
5. Issue books and create loan records.
6. Return books and calculate fines.
7. Maintain reservation queues.
8. Generate due-date and overdue notifications.
9. Process concurrent requests using synchronized operations.
10. Use wait() and notifyAll() for inter-thread communication.
11. Generate circulation and inventory-utilization reports.

Outputs

- Member records
- Book inventory
- Search results
- Issue/return confirmations
- Due dates
- Reservation queue
- Fine amount
- Notifications
- Circulation report
- Inventory-utilization report

- Exception messages
- Concurrent processing results

2. Objective and Expected Outcomes

2.1 Objectives

1. To apply object-oriented programming concepts to a practical library-management problem.
2. To implement encapsulation and data hiding.
3. To demonstrate inheritance and runtime polymorphism.
4. To use Java Collections Framework classes such as ArrayList, HashSet, HashMap and Hashtable.
5. To use generics, Iterator and ListIterator.
6. To implement built-in and user-defined exception handling.
7. To implement multithreading and thread priorities.
8. To maintain data consistency using synchronization.
9. To demonstrate inter-thread communication using wait() and notifyAll().
10. To generate useful circulation, fine and inventory reports.

2.2 Expected Outcomes

After completing the project:

- Library members and books can be managed efficiently.
- Available book copies are tracked accurately.
- Different membership types can have different borrowing limits and fine rates.
- Unavailable books can be reserved.
- Duplicate reservations are rejected.
- Overdue fines are calculated automatically.
- Due-date and overdue notifications are generated.
- Concurrent book-issue requests are safely handled.
- Invalid operations are handled without terminating the program.
- Library reports can be generated from the stored records.

3. Requirements, Constraints and Assumptions

3.1 Functional Requirements

ID	Functional Requirement
FR1	Register Student and Faculty members
FR2	Add and update book records
FR3	Display inventory and available copies
FR4	Search books
FR5	Issue books
FR6	Return books and calculate fines
FR7	Reserve unavailable books
FR8	Cancel reservations
FR9	Generate due-date notifications
FR10	Generate overdue notifications
FR11	Generate circulation reports
FR12	Generate inventory-utilization reports
FR13	Demonstrate concurrent book issue
FR14	Handle invalid inputs and exceptions

3.2 Non-Functional Requirements

- **Correctness:** Inventory should remain consistent.
- **Reliability:** Invalid operations should be handled safely.
- **Usability:** The system should provide a simple menu-driven interface.
- **Maintainability:** Different responsibilities should be separated into classes.
- **Performance:** In-memory collections should provide quick access for the assignment-scale data.
- **Safety:** Shared inventory operations must be synchronized.

3.3 Constraints

- Java console application.
- JDK 17 or later.
- No external database required.
- Data is maintained in memory.
- Notifications are simulated through console output.

- Designed for an academic-scale dataset.
- Thread priority is not used as a correctness mechanism.

3.4 Assumptions

- Every member has a unique member ID.
- Every book has a unique book ID.
- Student members can borrow up to 3 books.
- Faculty members can borrow up to 5 books.
- Standard loan period is 14 days.
- Student fine rate is Rs. 2 per overdue day.
- Faculty fine rate is Rs. 3 per overdue day.
- A reservation is required only when no copy is currently available.

4. Application of Relevant Course Knowledge / Concepts

The project directly applies the three mapped course outcomes.

Java Concept	Application
Classes and Objects	Member, Book, Reservation, Inventory, Notification and other classes
Encapsulation	Private attributes with controlled methods
Data Hiding	Internal state is not directly exposed
Inheritance	StudentMember and FacultyMember extend Member
Polymorphism	Membership-specific behaviour is selected at runtime
ArrayList	Loans, members and reservations
HashSet	Unique member collection
HashMap	Fast member and book lookup
Hashtable	Notification records
Generics	Type-safe collections
Iterator	Searching/traversing collections
ListIterator	Reservation traversal/removal
Exception Handling	Invalid input and library-specific exceptions
Multithreading	Concurrent issue and notification tasks

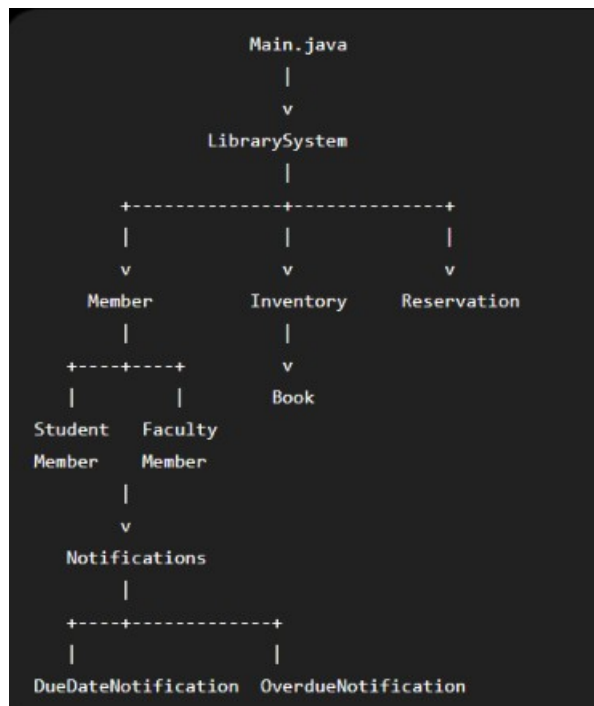
Java Concept	Application
Thread Priority	Different priorities assigned to concurrent tasks
Synchronization	Protects shared inventory
wait()	Waiting for book availability
notifyAll()	Notifies waiting threads after return

The rubric explicitly evaluates class design, inheritance/polymorphism, collections/generics/iterators, exception handling and multithreading/synchronization.

5. Design / Proposed Solution / Methodology

5.1 Proposed Architecture

The application follows an object-oriented architecture:



The system also contains separate thread classes:

ConcurrentIssueTask

AvailabilityWaitTask

OverdueNotificationTask

5.2 Major Classes

Class	Responsibility
Main	Menu-driven user interface
LibrarySystem	Controls the complete library workflow
Member	Base member class
StudentMember	Student-specific borrowing/fine behaviour
FacultyMember	Faculty-specific borrowing/fine behaviour
Book	Stores book information and stock
Inventory	Manages book inventory
Reservation	Represents a book reservation
Notification	Base notification class
DueDateNotification	Due-date reminder
OverdueNotification	Overdue message
ConcurrentIssueTask	Concurrent issue operation
AvailabilityWaitTask	Demonstrates waiting for availability
OverdueNotificationTask	Performs notification processing
LibraryExceptions	User-defined exceptions

6. Algorithm / Pseudocode / Flowchart

6.1 Main Algorithm

START

Load sample members and books

REPEAT

 Display main menu

 Read user choice

 IF choice = Register Member

 Validate input

 Create Student or Faculty member

 Store member

 ELSE IF choice = Add Book

Validate book details
Add book to inventory
ELSE IF choice = Display Inventory
 Display all books and available copies
ELSE IF choice = Search Book
 Search inventory
 Display matching books
ELSE IF choice = Update Book
 Find book
 Update required information
ELSE IF choice = Issue Book
 Validate member
 Validate book
 Check borrowing limit
 Check available copies
 Synchronize inventory
 Create loan
 Update member record

ELSE IF choice = Return Book
 Find active loan
 Calculate overdue days
 Calculate fine
 Update inventory
 Notify waiting threads
ELSE IF choice = Reserve Book
 Validate member and book
 Check availability
 Check duplicate reservation
 Add reservation to waitlist
ELSE IF choice = Cancel Reservation
 Search reservation

```

    Remove reservation using ListIterator
ELSE IF choice = Notifications
    Check active loans
    Generate due/overdue notifications
ELSE IF choice = Concurrent Demo
    Create issue threads
    Assign priorities
    Start threads
    Synchronize shared inventory
    Demonstrate wait/notifyAll
ELSE IF choice = Reports
    Generate circulation report
    Generate inventory-utilization report
ELSE IF choice = Exit
    STOP
    Handle exceptions
UNTIL Exit
END

```

6.2 Fine Calculation

Overdue Days = Return Date - Due Date

IF Overdue Days < 0

Overdue Days = 0

Fine = Overdue Days × Fine Rate

Example:

Due Date = 15-09-2026

Return Date = 17-09-2026

Overdue Days = 2

Fine Rate = Rs. 2/day

Fine = 2 × 2

= Rs. 4

6.3 Reservation Algorithm

Validate member

Validate book

IF available copies > 0

Reservation not required

ELSE

Search existing reservations

IF duplicate exists

Throw DuplicateReservationException

ELSE

Add reservation to waitlist

7. Implementation / Source Code and Environment / Tools Used

7.1 Software Environment

Component	Details
Programming Language	Java
JDK	JDK 17+
Tested Runtime	OpenJDK 21
IDE	Visual Studio Code
Interface	Console
Compiler	javac
Runtime	java
Diagram Tool	Graphviz
Version Control	Git / GitHub

7.2 Compilation

From the project root:

```
javac -d out src/*.java
```

7.3 Execution

```
java -cp out Main
```

7.4 Source Code Structure

```

src/
|
├─ Main.java
├─ LibrarySystem.java
├─ Inventory.java
├─ Member.java
├─ StudentMember.java
├─ FacultyMember.java
├─ Book.java
├─ Reservation.java
├─ Notification.java
├─ DueDateNotification.java
├─ OverdueNotification.java
├─ ConcurrentIssueTask.java
├─ AvailabilityWaitTask.java
├─ OverdueNotificationTask.java
└─ LibraryExceptions.java

```

8. Test Cases and Expected / Actual Results

Test ID	Test Case	Expected Result	Actual Result	Status
TC01	Display inventory	Books and copies displayed	Correct inventory displayed	PASS
TC02	Search Java book	Java book displayed	B101 displayed	PASS
TC03	Issue B101 to M001	Book issued and due date generated	Successful	PASS
TC04	Due notification	Reminder generated	Notification displayed	PASS
TC05	Return overdue B101	Fine calculated	Rs. 4.00 calculated	PASS
TC06	Reserve unavailable B102	Reservation added	Reservation added	PASS
TC07	Duplicate reservation	Exception displayed	Duplicate reservation rejected	PASS
TC08	Update B103	Book details updated	Updated record displayed	PASS
TC09	Invalid member	Exception handled	Invalid member message displayed	PASS
TC10	Concurrent issue	Only available copy issued	One success, one rejection	PASS
TC11	Waiting thread	Thread waits for availability	Thread resumed after return	PASS
TC12	Utilization report	Report generated	Report generated	PASS

9. Execution Screenshots / Outputs

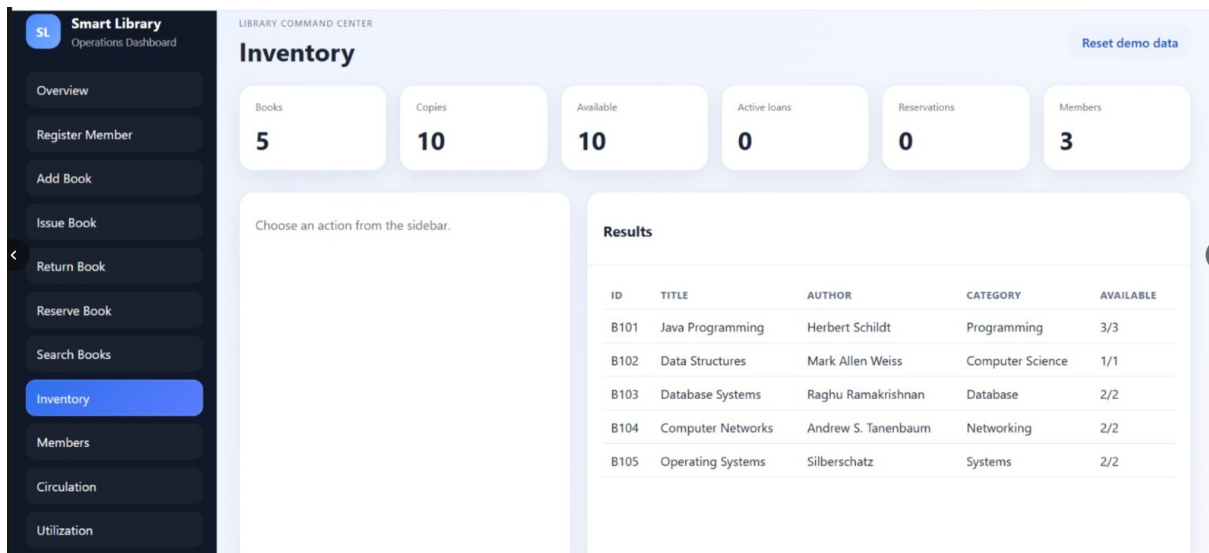


Figure 1: Displaying library inventory

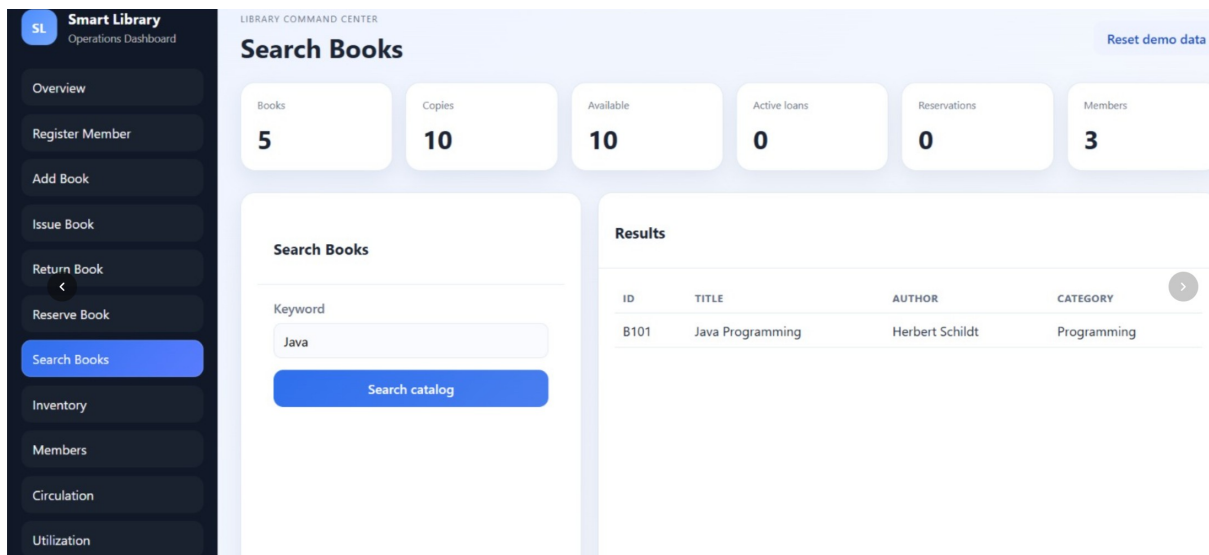


Figure 2: Search for books

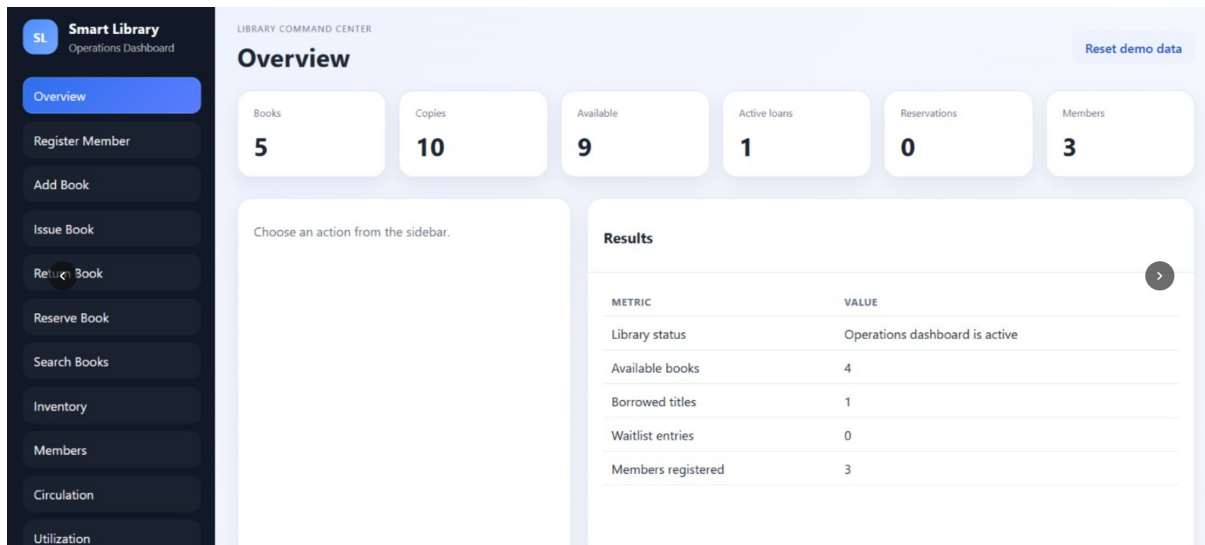


Figure 3: overview of dashboard.

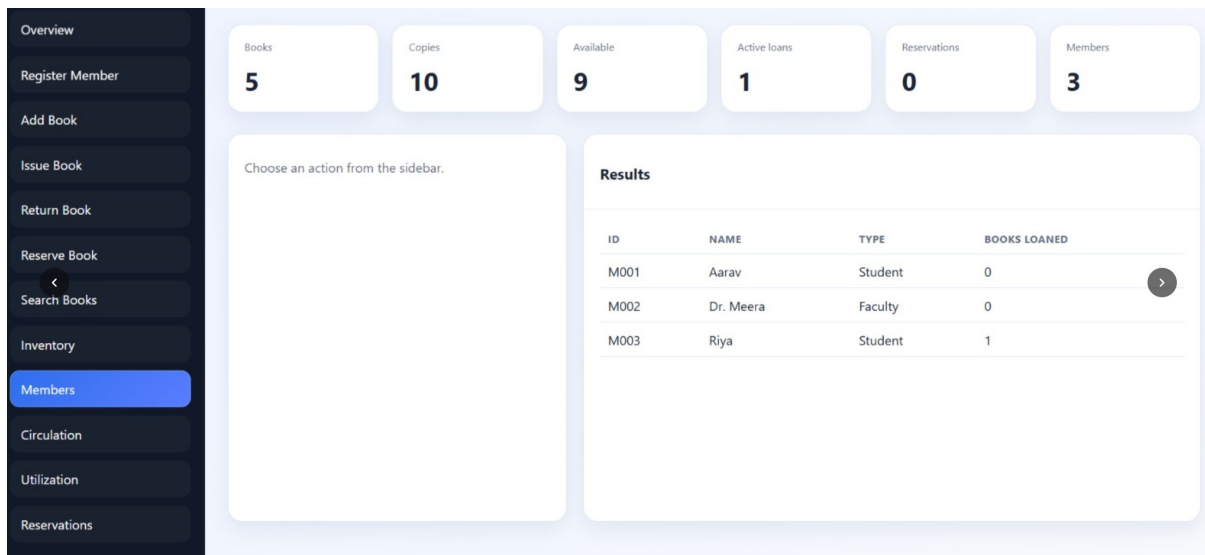


Figure 3: current registered students.

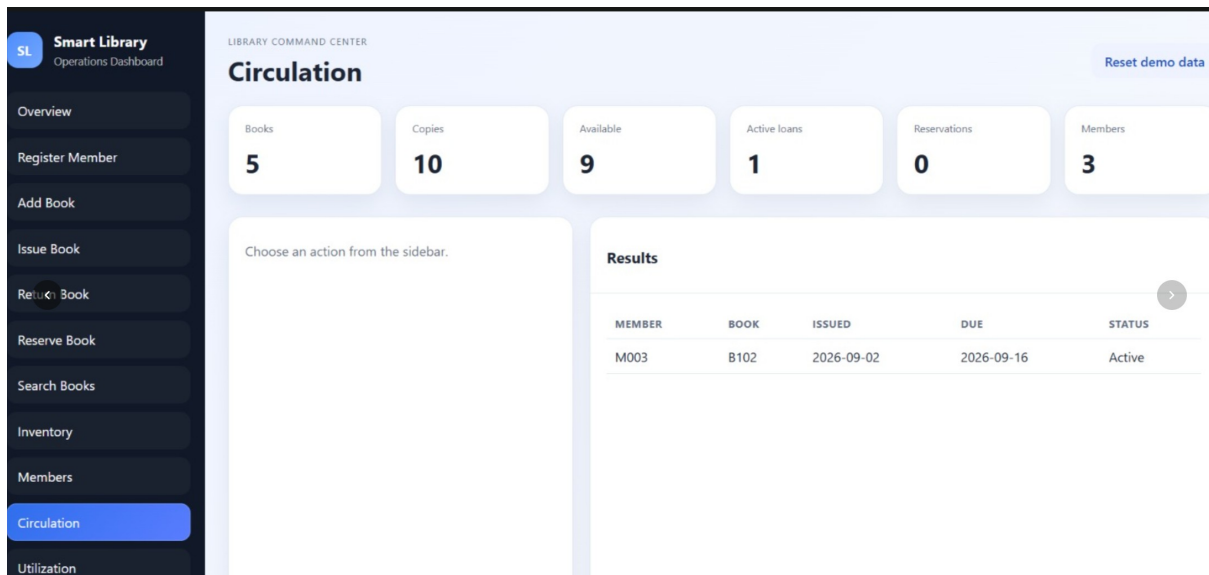


Figure 4: circulation .

10. Results and Validation

10.1 Functional Validation

Requirement	Evidence	Result
Member management	Member registration/listing	Satisfied
Book management	Add/update/search	Satisfied
Inventory management	Available-copy tracking	Satisfied
Book issue	Issue test	Satisfied
Book return	Return/fine test	Satisfied
Reservation	Waitlist test	Satisfied
Notification	Due/overdue test	Satisfied
Exception handling	Invalid input tests	Satisfied
Multithreading	Concurrent issue test	Satisfied
Synchronization	Shared inventory test	Satisfied
Inter-thread communication	Wait/notify test	Satisfied
Reports	Circulation/utilization reports	Satisfied

10.2 Fine Validation

Parameter	Value
Member	M001
Membership	Student
Book	B101
Due Date	15-09-2026
Return Date	17-09-2026
Overdue Days	2
Fine Rate	Rs. 2/day
Fine	Rs. 4.00

The calculated result matches the program output.

10.3 Concurrency Validation

For the one-copy book B102, two concurrent issue requests were generated.

Observed behaviour:

Thread 1 → SUCCESS

Thread 2 → Book unavailable

The available-copy count did not become negative.

The waiting task subsequently waited for B102 and resumed when the book was returned and notifyAll() was invoked.

This validates synchronization and inter-thread communication.

11. Analysis, Comparison, Trade-offs and Justification

11.1 Comparison of Approaches

Approach	Advantages	Limitations	Selection
Arrays	Simple	Fixed size and limited lookup	Not selected
Single ArrayList	Easy implementation	Less suitable for uniqueness and keyed lookup	Not selected
Multiple Java Collections	Flexible, efficient lookup and directly supports CO2	Slightly more complex	Selected
Database	Persistent and scalable	Requires additional setup and shifts focus away from core Java concepts	Future improvement

11.2 Trade-offs

In-memory storage

The current solution uses in-memory collections because they are simple and directly demonstrate Java Collections Framework concepts.

Trade-off: Data is lost when the program terminates.

Console notifications

Console output keeps the project self-contained.

Trade-off: It does not send real email or SMS notifications.

Synchronization

Synchronized methods provide a straightforward way to protect shared inventory.

Trade-off: Excessive synchronization can reduce concurrency, but for this assignment-scale system the simplicity and correctness are more important.

Thread Priority

Thread priorities demonstrate the required concept.

Trade-off: Thread priority does not guarantee exact execution order, so synchronization is used for correctness.

11.3 Final Solution Justification

The final design was selected because it satisfies the assignment's major technical requirements while remaining simple enough to compile and execute without external infrastructure.

It demonstrates:

- CO1 → OOP, encapsulation, inheritance and polymorphism.
- CO2 → Collections, generics and iterators.
- CO3 → Exception handling and multithreading.

12. Broader Considerations / SDG Relevance

SDG 4 – Quality Education

A properly managed digital library improves access to educational resources. Efficient searching, borrowing, reservations and reminders can help students and faculty access learning materials more effectively.

SDG 9 – Industry, Innovation and Infrastructure

The project demonstrates digital resource management using software engineering concepts such as object-oriented design, collections, concurrent processing and automated reporting.

SDG 16 – Peace, Justice and Strong Institutions

Accurate circulation records and reliable inventory management support transparent and accountable institutional processes.

Privacy

The current prototype uses sample data and does not send information to external services. A production system should implement authentication, authorization, secure data storage and appropriate privacy controls.

Professional Responsibility

Incorrect inventory management could result in unfair resource allocation. Therefore, synchronization and validation are important not only technically but also from a reliability and professional-responsibility perspective.

13. Conclusion, Limitations and Possible Improvements

13.1 Conclusion

The Smart Library Management, Book Reservation and Overdue Notification System successfully implements the major requirements of the CSA09 Java assignment.

The system combines:

- Object-oriented programming
- Encapsulation
- Inheritance
- Polymorphism
- Java Collections Framework
- Generics
- Iterators
- Exception handling
- Multithreading
- Thread priorities
- Synchronization
- Inter-thread communication
- Reservation management
- Fine calculation
- Notifications
- Reporting

Testing confirmed that the major operations work correctly and that concurrent access to shared inventory is handled safely.

13.2 Limitations

1. Data is stored only in memory.
2. There is no database.
3. Notifications are simulated through console output.
4. There is no authentication.
5. The interface is console-based.
6. Reports are text-based.
7. Thread priorities do not guarantee execution order.

13.3 Possible Improvements

Future versions can include:

- MySQL database using JDBC.
- User authentication.
- Role-based access control.
- Persistent member/book/loan records.
- Automatic reservation fulfilment.
- Email/SMS notifications.
- Web or mobile interface.
- Automated JUnit testing.
- Better search and filtering.
- Audit logs.
- Graphical reports.
- Configurable loan periods and fine rates.

14. Individual Contribution of Group Members

Reg. No.	Name	Responsibility	Percentage of Contribution
192324082	Mandati Sharan Mahi Reddy	Overall Java implementation, OOP class design, integration of modules, debugging and final testing	20%
192324084	Kandath Haneesh	Member and Book modules, encapsulation, inheritance, polymorphism, book search and update functionality	20%
192372075	O. Bhanu prakash Reddy	Java Collections Framework, generics, iterators, reservation and waitlist management	20%
192372222	Sri Hari	Exception handling, validation, fine calculation, due-date and overdue notification modules	20%
192524414	Antony Ajay	Multithreading, thread priorities, synchronization, inter-thread communication, reports and documentation	20%
Total			100%

15. One-Page Individual Reflection

Student Name: Kandath Haneesh

Registration No.: 192324084

This assignment gave me an opportunity to apply Java programming concepts to a realistic **Smart Library Management System**. My main contribution was developing and implementing the **Member and Book modules**, along with applying **object-oriented programming concepts such as encapsulation, inheritance and polymorphism**. I also worked on the **book search and update functionality**, which helped me understand how individual modules can be designed to perform specific tasks within a complete system.

I contributed to the design and implementation of the **Member and Book classes** using encapsulation and data hiding. I used appropriate attributes and methods to ensure that data could be accessed and modified in a controlled manner. This helped me understand the importance of protecting object data and maintaining a clear structure within a Java application.

I also applied **inheritance and polymorphism** while designing different types of members. Inheritance helped me reuse common properties and behaviours, while polymorphism allowed different member types to provide their own implementations where required. Through this, I gained a better understanding of how OOP principles can make a system more flexible, reusable and maintainable.

Another important part of my contribution was implementing **book search and update functionality**. The search feature allows users to find books based on relevant information, while the update functionality allows book details such as title, author, category or availability to be modified when necessary. Implementing these features improved my understanding of **Java Collections, conditional logic, object manipulation and data validation**.

I also tested the Member and Book modules using different valid and invalid inputs. Debugging errors and checking different scenarios helped me understand the importance of **exception handling, validation and systematic testing**. This ensured that the modules worked correctly and produced reliable results during different operations.

The project relates to **SDG 4 – Quality Education** by making educational resources easier to manage and access, and **SDG 9 – Industry, Innovation and Infrastructure** by demonstrating how digital systems can improve resource management in libraries.

If given additional time, I would improve these modules by adding **database connectivity, advanced book-search filters, authentication, a graphical/web interface and improved validation**. These improvements would make the system more practical and user-friendly.

Overall, this assignment helped me attain **CO1** through the practical application of **encapsulation, inheritance and polymorphism**, and **CO2** through the use of **Java Collections and book search/update operations**. My key learning was understanding how **OOP principles can be applied to design reusable Member and Book modules and how search and update functionality can be integrated to create an effective library management system**.

Signature: Kandath Haneesh

Date: 02-09-2026

16. References

1. *Common Course Assignment Format*, supplied by the Department.
2. *CSA09 – Programming in Java (SLOT A): Smart Library Management, Book Reservation and Overdue Notification System using Java*, supplied assignment question.
3. Oracle Java documentation – Java Language and Standard Library concepts.
4. OpenJDK documentation – Java Development Kit and runtime environment.
5. Graphviz documentation – architecture and flowchart generation.
6. Project sample dataset – data/sample_data.txt.
7. Project test cases – data/test_cases.csv.
8. Java source code developed for the assignment.
9. AI-assisted development/documentation tools, where applicable, acknowledged according to institutional academic-integrity requirements.